Put your app on a server

A step-by-step guide. No technical background needed. Allow about 30 minutes.

Before you start

Have these ready:

- A login for AWS (or another cloud provider).
- The link to your app on GitHub.
- The app's env file (its settings). Ask a developer if you don't have it.
- **Optional:** a Gitleaks license key, if you want secret scanning in automatic deploys.
- **Optional:** a domain name, and access to where it is managed.

Part 1: Create the server

These steps are for AWS. Other providers are listed at the end of this part.

1. Sign in to AWS. In the top-right corner, choose the region closest to your users.
2. Search for **EC2** at the top of the page and open it. Click **Launch instance**.
3. **Name:** type something clear, such as *my-app-dev*.
4. **Application and OS Images:** choose **Ubuntu**, then **Ubuntu Server 24.04 LTS**.
5. **Instance type:** use the table below. The Architecture must match it.
6. **Key pair:** click **Create new key pair**, give it a name, and create it. Keep the downloaded **.pem** file somewhere safe.
7. **Network settings:** click **Edit** if needed, and create a security group with these rules:

Type	Port	Source
SSH	22	0.0.0.0/0 (Anywhere)
HTTP	80	0.0.0.0/0 (Anywhere)
HTTPS	443	0.0.0.0/0 (Anywhere)

8. **Storage:** set it to **30 GB**.
9. Click **Launch instance** and wait until its state says **Running**.

Which instance type?

Instance type	Architecture to choose	Good for
t4g.medium	64-bit (Arm)	Most apps. Lower cost.
t3.medium	64-bit (x86)	Most apps. Use this if an app has trouble on Arm.
t4g.small	64-bit (Arm)	Small, simple apps only.

Give the server a permanent address

Without this, the server's address changes if it is stopped, and your domain and automatic deploys stop working.

1. In EC2, open **Elastic IPs** in the left menu and click **Allocate Elastic IP address**, then **Allocate**.
2. With the new address selected, click **Actions**, then **Associate Elastic IP address**.
3. Choose your instance and click **Associate**.

Using a different cloud provider?

The steps are similar. Create a server with Ubuntu 24.04, about 2-4 GB of memory and 30 GB of disk, open ports 22, 80 and 443 to everyone, and give it a fixed IP address.

Provider	The server is called	Fixed IP is called
Google Cloud	Compute Engine VM	Static external IP
Microsoft Azure	Virtual Machine	Static public IP
DigitalOcean	Droplet	Reserved IP

Part 2: Create a GitHub token

A token is like a temporary password that lets the setup get your code. It is only used while the setup runs and is not saved on the server.

1. On GitHub, click your profile picture, then **Settings**.
2. At the bottom of the left menu, click **Developer settings**.
3. Click **Personal access tokens**, then **Fine-grained tokens**, then **Generate new token**.
4. Give it a name and set the expiry to **7 days**.
5. **Resource owner:** choose the company or account that owns the repo.
6. **Repository access:** choose **Only select repositories** and pick your repo.
7. Under **Permissions**, add these three and set each one to **Read and write**: **Contents**, **Secrets**, **Workflows**.
8. Click **Generate token** and copy it straight away. GitHub shows it only once.

Company repos: an organization owner may need to approve the token before it works.

Part 3: Run the setup

1. In EC2, open **Instances**, select your server and click **Connect**.
2. Open the **EC2 Instance Connect** tab, make sure the username is **ubuntu**, and click **Connect**. A black terminal window opens in your browser.
3. Copy the command below, paste it into that window, and press **Enter**.

```
bash <(curl -fsSL https://raw.githubusercontent.com/jaideepp247/server-setup/main/setup.sh)
```

Pasting in the terminal: use **Ctrl+Shift+V** on Windows, or **Cmd+V** on Mac. Passwords and tokens stay invisible while you paste them. That is normal: paste once and press Enter.

Answer the questions as they appear. Here is what happens:

Step	What it does	What you do
1. Install	Installs Node 24, npm, yarn and pm2.	Just wait a few minutes.
2. Get code	Downloads your app from GitHub.	Paste the repo link and token. Type the branch, such as **dev**, or press Enter for the main branch.
3. Settings	Saves your env file.	Paste the whole file, press Enter, type **END**, press Enter. Check the list of names and press Enter.
4. Build	Finds the right commands and builds the app.	Press Enter to accept the commands. Wait.
5. Start	Starts the app, finds its port and checks that it works.	Just wait. If it fails, you'll see the error and can paste the env file again.
6. Auto deploy	Asks if you want automatic deploys.	Type **y** or **n**. For yes: paste the Gitleaks key or press Enter to skip, then press Enter to let it create the SSH key.
7. Pipeline	Adds the deploy pipeline and secrets to GitHub.	Just wait.
8. Domain	Connects your domain with HTTPS.	Type **y** or **n**. For yes: type the domain, then add the record it shows you at your domain provider.

When you see **All done!**, your app is live. The screen shows its address.

No domain? The app runs, but people can't open it yet, because only ports 80 and 443 are open. Run the setup command again later to connect a domain.

If something goes wrong

It is always safe to run the setup command again. It updates the existing app and doesn't duplicate anything.

What you see	What to do
This server is not Ubuntu	Create a new server and choose Ubuntu in Part 1.
GitHub refused access	Check the repo link. Check that the token hasn't expired and that it includes this repo.
The branch doesn't exist	Check the branch name on GitHub. Capital letters matter.
The app is not working yet	Read the message under it. Usually an env value is wrong or missing. Choose option 1 and paste the corrected env file.

What you see	What to do
Ran out of memory	Your server is too small. Create a larger one, such as t3.medium.
This token can't manage this repo's secrets	Go back to Part 2 and check all three permissions. Create a new token and paste it.
The domain isn't set up yet	Add the A record exactly as shown, wait a few minutes, and press Enter to check again.
The domain isn't answering yet	Check that ports 80 and 443 are open (Part 1). If you use Cloudflare, set the record to DNS only (grey cloud).
The terminal window closed	Reconnect (Part 3) and run the command again.
Anything else	Send a developer the error on screen and the file **server-setup.log** from the home folder.

Everyday commands

Connect to the server (Part 3) and type one of these, replacing *my-app* with your repo's name:

Command	What it does
`pm2 status`	Shows whether the app is running.
`pm2 logs my-app`	Shows the app's messages. Press Ctrl+C to leave.
`pm2 restart my-app`	Restarts the app.

Finished with a server? Terminate the instance and release its Elastic IP in AWS, so you stop paying for them.